Presentation Script

Word-for-word speaking notes for all 13 slides — NexCare_Capstone_Presentation.pptx

Slide-by-Slide Script | Timing Guide (~12 min) | Anticipated Questions | Delivery Tips

How to use this document

Each slide below has four parts: **What to Say** (read this almost word-for-word until you're comfortable, then put it in your own words), **Key Points to Hit** (don't skip these even if you reword the script), **If Asked** (likely follow-up questions with short answers), and a timing estimate. Total deck run-time target: **10–13 minutes**, leaving time for Q&A. Practice out loud at least twice before presenting — reading silently is not the same as speaking it.

1 Cover — NexCare

~40 sec

WHAT TO SAY

"Good morning/afternoon everyone. My name is Rishitha Manyam, and today I'll be presenting my capstone project for the Wipro RGA Program 2025 — it's called NexCare, an Interactive Dashboard System for Customer Service Operations. This project was built using the Java Full Stack technologies I learned during the program — specifically Angular on the frontend and Spring Boot on the backend."

KEY POINTS TO HIT

- State your name and the program name clearly — this is the formal opening.
- Say the project name "NexCare" and its one-line description once, clearly.
- Mention it's a Java Full Stack project — sets audience expectations for the tech questions ahead.

Pause here, make eye contact, then click to Slide 2.

2 Agenda

~25 sec

WHAT TO SAY

"Here's what I'll cover today — I'll start with the problem this project solves, then walk through my solution and tech stack, explain the system architecture and security model, show you the database design, walk through the key features, talk about challenges I faced, and close with what I learned. I'll leave time at the end for questions."

KEY POINTS TO HIT

- Just a quick read-through of the bullet list — don't over-explain, this slide is a map, not content.

3 Problem Statement

~50 sec

WHAT TO SAY

"Let's start with the problem. Telecom customer service teams in India today rely on disconnected tools. Representatives use spreadsheets to track tickets, which means there's no single, unified view of what's happening with any customer. Managers have no real-time way to see how their team is performing — no visibility into who's overloaded or who's resolving tickets quickly. Customers themselves have no way to track their own complaint status, so they end up calling back repeatedly just to ask 'what's the status of my ticket?' — which wastes everyone's time. And when there's a service outage in an area, there's no targeted way to inform just the customers who are actually affected."

KEY POINTS TO HIT

- Spreadsheets → no unified view
- Managers → no real-time visibility
- Customers → no self-service tracking, leads to repeat calls
- Outages → no targeted communication

IF ASKED

"Is this based on a real company?" → "It's modeled on common pain points reported across small and mid-sized Indian telecom service desks — it's a representative problem, not one specific company's internal system."

4 Solution Overview — NexCare

~60 sec

WHAT TO SAY

"My solution is NexCare — a full-stack web application purpose-built for Indian telecom customer service operations. It gives every user role a dashboard tailored to exactly what they need. The entire API is secured with JWT tokens, so every user only ever sees the data they're authorised to see — a customer cannot see another customer's tickets, and a representative cannot see another representative's workload. Ticket management happens in real time with a full audit trail — every status change is timestamped. There's also a live chatbot that can answer twelve common customer questions instantly without needing a human representative. For visibility, I built analytics charts — bar charts and doughnut charts — so managers can see performance at a glance. Outage alerts are smart — they only appear to customers in the affected city, not everyone. And the whole application is localised for India — INR currency, Indian city names, Indian phone number formats. Finally, there's zero setup required for the database — it seeds all its own demo data automatically the first time it runs."

KEY POINTS TO HIT

- Four role-based dashboards
- JWT security — strict data isolation
- Real-time tickets with audit trail
- Chatbot + analytics charts
- City-targeted outage alerts
- India localisation + zero-setup database

IF ASKED

"What does 'full audit trail' mean exactly?" → "Every ticket records when it was created, when a representative first responded, and when it was resolved — these timestamps are never entered manually, they're captured automatically by the system the moment the status changes."

5 Tech Stack

~70 sec

WHAT TO SAY

"On the frontend, I used Angular 17 with standalone components, meaning there's no NgModule anywhere in the application — that's the modern Angular approach. Everything is written in strict TypeScript. For visual analytics I used Chart.js for the bar and doughnut charts. Styling is plain CSS3 with a lilac theme and smooth cubic-bezier animations. Routing uses Angular Router with lazy loading, so a dashboard's code only downloads when a user actually visits it, plus hash-based URLs for easy hosting. I used RxJS for reactive state management and for communication between components that aren't directly related. And there's a proxy configuration so the Angular dev server forwards any '/api' request straight to the backend on port 8080 — no CORS headaches during development. On the backend, it's Spring Boot 3.2.5 powering the REST API. Spring Security handles the JWT filter chain. I used the JJWT library, version 0.12.3, for generating and validating tokens. Spring Data JPA is the ORM layer for all my entities. The database is H2, running in file mode, so no external database server is needed at all. Passwords are hashed using BCryptPasswordEncoder — never stored in plain text. And the whole project is built and managed with Maven."

KEY POINTS TO HIT

- Frontend: Angular 17 standalone, TypeScript, Chart.js, RxJS, lazy loading
- Backend: Spring Boot, Spring Security, JJWT, Spring Data JPA, H2, BCrypt
- Say "standalone components, no NgModule" — this signals you know modern Angular, not textbook Angular

IF ASKED

"Why Angular over React?" → "Angular is a complete framework — routing, forms, and the HTTP client are all built in. React is just a library, so you'd need to add several third-party packages to get the same coverage. For an enterprise-style dashboard like this, Angular's structure made more sense."

"Why H2 and not MySQL/PostgreSQL?" → "H2 needed zero installation — it runs as a file right inside the project, which made the application instantly runnable for demo and evaluation purposes. For a real production deployment, I'd migrate to PostgreSQL, which I mention in my future scope."

6 System Architecture

~60 sec

WHAT TO SAY

"NexCare follows a three-tier architecture. At the top is the Presentation layer — that's the Angular 17 single-page application, handling all the components, routing, interceptors, and guards. Below that is the Service or API layer — the Spring Boot REST controllers for Authentication, Users, Tickets, Analytics, and Outages. At the bottom is the Data layer — Spring Data JPA on top of the H2 file database, storing Users, Tickets, Outages, and Notifications. These layers talk to each other purely over HTTP using JSON. During development, the proxy I mentioned handles the connection; in production it would be direct CORS-enabled communication. And critically — every single request between the frontend and backend carries a JWT bearer token, so authentication is checked on every call, not just at login."

KEY POINTS TO HIT

- Three tiers: Presentation (Angular) → Service/API (Spring Boot) → Data (JPA + H2)
- Communication: HTTP/JSON, proxy in dev, CORS in prod
- JWT bearer token on every request — not just login

IF ASKED

"Why three tiers instead of putting everything in one app?" → "Separating frontend and backend means they can be deployed, scaled, and even rewritten independently. It's also the standard pattern in the industry for any application that might need a mobile client later — the same backend API could serve a mobile app without any changes."

7 Role-Based Dashboards

~55 sec

WHAT TO SAY

"There are four roles in NexCare, each with its own dedicated dashboard. The Admin manages all employees, can view every ticket in the system, reports service outages, and sees an overall analytics view. The Manager monitors their team's tickets, resolves outages, views individual representative performance, and manages escalations. The Representative views and updates only their assigned tickets, can search for customers, and tracks their own resolution time. And the

Customer can raise new tickets, track their status, rate the support they received, view outage alerts for their city, and use the chatbot."

KEY POINTS TO HIT

- Four roles, four dashboards — not one dashboard with toggled visibility
- Each role's permissions map directly back to the problem statement

IF ASKED

"What stops a Representative from just changing the URL to access the Admin page?" → "Two layers stop that — first, an Angular route guard checks the role stored from the JWT and redirects them immediately. Second, even if they bypassed the frontend entirely and called the API directly, the backend's @PreAuthorize checks would reject the request with a 403 — the frontend guard is just a convenience, the backend is the real enforcement."

8 Security Implementation — JWT

~90 sec (most technical slide)

WHAT TO SAY

"This is the most important technical slide, so let me walk through the full flow step by step. One — the user submits their email and password through the Angular Sign In form. Two — my AuthController calls a UserDetailsService, which uses BCrypt to verify the submitted password against the hashed password stored in the database — the plain password is never stored anywhere. Three — once verified, my JwtService generates a signed token containing the user's ID, email, role, and a 24-hour expiry. Four — that token is returned to the frontend and stored in sessionStorage, which automatically clears when the browser tab closes — that's a deliberate security choice over localStorage. Five — from that point on, every single API call from the frontend goes through a JWT interceptor that automatically attaches the token as an 'Authorization: Bearer' header — I never have to manually add it in any service. Six — on the backend, my JwtAuthFilter validates that token's signature on every protected endpoint before the request is even allowed to reach the controller. Seven — on the frontend, an Auth Guard checks the role stored from that token before allowing the Angular router to render a protected page. And eight — when the user logs out, sessionStorage is cleared, so even if that token were somehow intercepted, it becomes unusable immediately."

KEY POINTS TO HIT

- Walk through all 8 steps in order — this is the slide evaluators will probe hardest
- Emphasize: password never stored in plaintext (BCrypt)
- Emphasize: sessionStorage over localStorage was a deliberate choice
- Emphasize: security enforced at BOTH frontend (guard) AND backend (filter) — two independent layers

IF ASKED

"What's inside the JWT token exactly?" → "Three parts: a header describing the signing algorithm, a payload with the user's ID, email, role, and expiry timestamp, and a signature — all three are Base64-encoded and joined with dots. The signature is what prevents anyone from tampering with the payload, because regenerating a valid signature requires a secret key only the server has."

"Why sessionStorage instead of localStorage?" → "localStorage persists even after the browser is closed, which is a bigger attack surface if the device is shared. sessionStorage clears the moment the tab closes, which is safer for a demo/internal tool context."

"What happens after the token expires?" → "Any further API call returns 401 Unauthorized, and the frontend interceptor catches that and redirects the user back to the login page to re-authenticate."

9 Database & Data Flow

~65 sec

WHAT TO SAY

"For data, I'm using an H2 file database that persists across restarts — so demo data isn't lost every time the server stops. There are four main tables. Users — all four roles live in one table, distinguished by a role enum, which keeps authentication logic simple. Tickets — storing the customer ID, assigned representative ID, status, domain, and customer rating. Outages — storing location, affected areas, domain, and severity. And Notifications — storing the recipient's user ID, the message, a read flag, and when it was created. All of this is seeded automatically on startup through a DataInitializer class using Spring's @PostConstruct annotation — by default it loads 11 demo users, 10 tickets, 2 outages, and 4 notifications, so the app is immediately demo-ready. On the API side, here are the core endpoints: POST /api/auth/login returns a JWT token. POST /api/auth/register creates a new customer account. GET /api/tickets returns tickets already filtered by the caller's role — a customer only gets their own, a representative only gets their assigned ones. PUT /api/tickets/{id} updates status and automatically stamps the audit fields. GET /api/analytics/{role} returns an aggregated statistics map for dashboards. And GET /api/outages/location filters outages down to just one city for the customer view."

KEY POINTS TO HIT

- 4 tables: Users (single table, role enum), Tickets, Outages, Notifications
- Auto-seeding via @PostConstruct — emphasize "zero manual setup"
- Role-based filtering happens server-side in the service layer, not just hidden in the UI

IF ASKED

"Why one Users table for all roles instead of separate tables?" → "All roles share the same core fields — name, email, password, phone, location — so one table with a role enum column avoids duplicate schema and makes authentication a single, simple query regardless of who's logging in."

"Does the seed data get duplicated if I restart the server multiple times?" → "No — DataInitializer checks if the admin account already exists before seeding anything, so restarts are safe and idempotent."

10 Key Features Walkthrough

~75 sec (pairs well with a live demo)

WHAT TO SAY

"Let me highlight ten features that I think best show the depth of this project. The login page has a fully animated full-page lilac overlay transition between Sign In and Sign Up, built with CSS cubic-bezier easing. Every dashboard route is lazy-loaded — Angular only downloads a dashboard's code when a user actually navigates to it. Tab navigation inside dashboards uses URL query parameters, so every tab is bookmarkable and survives a page refresh. Analytics are powered by live Chart.js bar charts for tickets by domain and doughnut charts for status breakdown. The outage alert banner is smart — it only appears to customers whose city actually matches the outage location, nobody else sees it. There's an AI chatbot recognising twelve different intent keywords, with auto-scroll, and it can even be triggered from the sidebar using an RxJS Subject as an event bus. Customers can rate closed tickets, and that score immediately shows up on the representative's own dashboard. Representatives can search customers by name, phone, or ID, with a 400-millisecond debounce so we're not spamming the API on every keystroke. First-time representative logins are detected and they're prompted to change their password before doing anything else. And the whole thing is localised for India — Rupee currency symbols, Indian city names, and Indian phone number formatting."

KEY POINTS TO HIT

- If you have time for a live demo, this is the slide to pause on and actually show 2-3 of these live
- Debounce and lazy-loading show performance awareness, not just feature-completeness
- City-matched outage banner reinforces the "Solution Overview" promise from slide 4

IF ASKED

"Is the chatbot using a real AI model?" → "No — it's a keyword/intent matching system covering 12 common questions like 'how do I raise a ticket' or 'what's my ticket status'. It's instant and doesn't need an external API, which fits a self-contained capstone project. I've noted upgrading it to an NLP-based model as future scope."

11 Challenges & Solutions

~70 sec

WHAT TO SAY

"No project goes perfectly the first time, so let me share five real challenges I ran into. First — the sidebar and the chatbot are sibling components with no direct parent-child relationship, so they couldn't easily talk to each other; direct DOM manipulation kept getting overridden by Angular's own change detection. I solved this by creating a shared ChatService using an RxJS Subject as an event bus. Second — I originally used scrollIntoView for tab navigation, but it wasn't reliable across page refreshes. I switched entirely to Angular Router query parameters instead, which is reliable across refreshes and roles. Third — Chart.js needs an actual canvas element in the DOM before it can draw, but my charts were inside Angular @if blocks that hadn't rendered yet. I fixed this with a short setTimeout after the tab switch to make sure the DOM element existed first. Fourth — Angular's dev server on port 4200 and Spring Boot on port 8080 triggered CORS errors during development. I added a proxy.conf.json so the dev server transparently forwards all /api requests, eliminating the cross-origin issue entirely. And fifth — my database seed was re-running every time I restarted the server. I fixed that by adding an existsById check in DataInitializer so it only seeds data if the admin account doesn't already exist."

KEY POINTS TO HIT

- This slide is your chance to prove you debugged real problems, not just followed a tutorial
- Each challenge should have a crisp one-line "why it broke" + "what fixed it"

IF ASKED

"What was the hardest bug to fix?" → "Honestly the Chart.js timing issue — it wasn't an error message, the chart would just silently not appear, so it took a while to realise the canvas element didn't exist yet when the chart code ran."

12 Key Learnings & Conclusion

~65 sec

WHAT TO SAY

"This project taught me a number of things I couldn't have fully learned from coursework alone. I learned how Spring Security combined with JWT enables stateless authentication that can scale — no server-side session storage required. I learned Angular's standalone component model, which removes all the NgModule boilerplate that older Angular tutorials are full of. I learned that RxJS Subjects are the correct pattern for communication between components that aren't directly related. I learned to write functional interceptors and guards, which is the modern Angular 17 style rather than the older class-based approach. I learned how a proxy configuration bridges frontend and backend cleanly during development. I learned that Chart.js has to respect Angular's own rendering lifecycle, not fight against it. And I learned to rely on @PostConstruct for dependable application startup behaviour. To conclude — NexCare demonstrates a production-grade Java Full Stack application. It's secure, role-based, and real-time, purpose-built for Indian telecom operations. It follows a clean architecture — an Angular single-page application, a Spring Boot REST API, and an H2 database. It requires zero manual setup, running with just two commands after installing the prerequisites. And it covers the full software development lifecycle — design, development, security, testing, and documentation. I've also prepared comprehensive supporting guides covering setup, a technical Q&A reference, and a full code walkthrough."

KEY POINTS TO HIT

- Tie learnings back to specific challenges from slide 11 if you have time — shows reflection, not just a list
- End on the "zero manual setup, two commands" line — strong, concrete closing fact

13 Thank You / Q&A

~15 sec + open floor

WHAT TO SAY

"Thank you for your time. That concludes my presentation on NexCare. I'd be happy to take any questions, and I'm also glad to walk through a live demo or any part of the code in more detail."

KEY POINTS TO HIT

- Smile, slow down, make eye contact — this is the line people remember

- Offering a live demo proactively signals confidence — only say it if you're ready to actually do it

Final Pre-Presentation Checklist

- ☐ Practiced the full script out loud at least twice, end to end, with a timer
- ☐ Backend running locally and confirmed reachable (login test with admin@csd.com works)
- ☐ Frontend running locally at http://localhost:4200 and confirmed loading
- ☐ Demo credentials memorised or written on a sticky note: admin@csd.com / admin123
- ☐ Laptop charger plugged in, brightness up, notifications silenced before presenting
- ☐ PPTX opened and tested in Presenter View at least once beforehand
- ☐ Read through the Technical Q&A Guide (docs/02) the night before for extra confidence
- ☐ Know which 2-3 features you'll demo live on slide 10 if time allows